Mathematical Background of the Duotrigordle Solver

Abstract

This note describes the model and the decision rule of the solver (`duotrigordle_solver.py`) and of the live bridge (`duo_bridge_server.py`). The solver keeps, per board, the exact set of words from its solution list that are consistent with the observed feedback, puts a heuristic prior on each set, and chooses every guess greedily by minimising the expected logarithmic size of the candidate sets after the guess. With the default settings, boards whose target is already determined are cleared first.

Contents

1 Game model **1**
1.1 Feedback function 2

2 State and candidate sets **2**
2.1 Fallback for missing targets 3

3 Prior distribution per board **3**

4 Scoring a guess **4**
4.1 Per-board feedback distribution 4
4.2 Relation between ELog and information gain 4
4.3 Candidate features 5

5 Decision rule **5**

6 Bounds on the number of guesses **6**

7 Computational cost **6**

8 Limitations **6**

1 Game model

Let Σ be the alphabet and $L = 5$ the word length. We use three word lists:

- $\mathcal{A} \subseteq \Sigma^L$, the solution list,
- $\mathcal{G} \subseteq \Sigma^L$, the allowed guesses (`guesses_en.txt`),
- $\mathcal{V} = \mathcal{G} \cup \mathcal{A}$, all valid words.

In the standalone simulation $\mathcal{A}$ is the file `solutions_en.txt`. The bridge server uses the union of `solutions_en.txt` and `solutions_learned.txt` (targets learned from earlier games), or $\mathcal{A} = \mathcal{G}$ with `--wide-solutions`. The sizes $|\mathcal{A}|$ and $|\mathcal{G}|$ depend on these files; they are treated as constants here and printed by the server at startup (`Lösungen=...`, `Guesses=...`).

A game consists of $M = 32$ boards (`NUM_BOARDS`) with pairwise distinct target words $t_1, \dots, t_M$ and at most $K = 37$ guesses (`MAX_GUESSES`). Every guess g_k is applied to all boards that are not yet solved. In the simulation the targets are drawn from $\mathcal{A}$; on the website they come from a list that is not known to the solver (see Section 2.1).

The solver only proposes words from $\mathcal{G}' = \mathcal{G} \setminus \mathcal{X}$, where $\mathcal{X}$ is the set of words the website has rejected (field `exclude` of a bridge request; $\mathcal{X} = \emptyset$ in the simulation). A target outside $\mathcal{G}'$ can therefore never be entered by the solver.

1.1 Feedback function

Definition 1 (Wordle feedback)**.** For a guess g and a target t, the feedback $f(g,t) \in \{0,1,2\}^L$ (grey, yellow, green) is computed in two passes. First,

$$f_i(g,t) = 2 \quad \text{if } g_i = t_i.$$

Let $r(c) = \#\{\, j : t_j = c,\ g_j \neq t_j \,\}$ be the number of letters c in t that are not used up by green positions. Then, going through the non-green positions $i = 1, \dots, L$ from left to right,

$$f_i(g,t) = \begin{cases} 1 & \text{if } r(g_i) > 0, \text{ and afterwards } r(g_i) \leftarrow r(g_i) - 1, \\ 0 & \text{otherwise.} \end{cases}$$

The definition handles repeated letters exactly. For example, $f(\texttt{eerie}, \texttt{there}) = (1,0,1,0,2)$: the target has two `e`s, one is used by the green last position, so only the first of the remaining guessed `e`s turns yellow.

Feedback is encoded as an integer $\kappa(\varphi) = \sum_{i=1}^{L} \varphi_i \, 3^{\,i-1} \in \{0, \dots, 242\}$. The all-green pattern $\mathbf{2} = (2,2,2,2,2)$ has the code $\kappa(\mathbf{2}) = 242$.

Lemma 1 (Vectorised yellow rule)**.** *For a non-green position i let*

$$a_i = \#\{\, j : t_j = g_i \,\} - \#\{\, j : t_j = g_i,\ g_j = t_j \,\}, \qquad u_i = \#\{\, j < i : f_j = 1,\ g_j = g_i \,\}.$$

Then $f_i(g,t) = 1 \iff u_i < a_i$.

Proof. a_i equals the initial value of $r(g_i)$, and u_i counts how often $r(g_i)$ has already been decremented before position i. So $r(g_i) > 0$ at step i exactly when $a_i - u_i > 0$. □

Lemma 1 lets `numpy_feedback_codes` and `cupy_feedback_codes` compute a full matrix of feedback codes for B guesses against N targets with array operations of size $B \times N \times L$.

2 State and candidate sets

After k guesses with observed feedback $\varphi_{b,1}, \dots, \varphi_{b,k}$ on board b, the candidate set is

$$C_b^{(k)} = \{\, t \in C_b^{(0)} : f(g_j, t) = \varphi_{b,j} \text{ for all } j \le k \,\}, \qquad C_b^{(0)} = \mathcal{A}.$$

It can be updated incrementally: $C_b^{(k)} = \{t \in C_b^{(k-1)} : f(g_k, t) = \varphi_{b,k}\}$. Board b is *solved* at step $\tau_b = k$ if $\varphi_{b,k} = \mathbf{2}$. The set of active boards is $\mathcal{B} = \{b : b \text{ not solved}\}$. A board with $|C_b| = 1$ is called a *singleton*: if $t_b \in C_b$, its target is known, but it still costs one guess to enter it.

Since $f(w,w) = \mathbf{2}$ for every word w, a guess g_k is removed from C_b on every active board b with $\varphi_{b,k} \neq \mathbf{2}$. In particular, the word that solves one board never remains a candidate on any other board.

2.1 Fallback for missing targets

The bridge server allows the website to use a solution list different from $\mathcal{A}$. If the update yields $C_b^{(k)} = \emptyset$ for the first time on board b, the server rebuilds this board alone from all valid words:

$$C_b^{(k)} = \{\, t \in \mathcal{V} : f(g_j, t) = \varphi_{b,j} \text{ for all } j \le k \,\}.$$

Later updates of this board filter the rebuilt set as usual. If the website's targets are contained in $\mathcal{V}$, the rebuilt set always contains t_b. If it is empty (or a rebuilt board becomes empty again), the server reports an error; then either the observed feedback is wrong (for example a misread colour) or $t_b \notin \mathcal{V}$. The other boards keep the smaller list $\mathcal{A}$, so the cost of the fallback is paid only where it is needed. The fallback only triggers once C_b is empty; as long as a wrong list $\mathcal{A}$ still contains words consistent with the feedback, C_b may not contain t_b. The standalone simulation has no fallback, because its targets lie in $\mathcal{A}$.

3 Prior distribution per board

The candidates of a board are not treated as equally likely. For each board, the function `compute_board_model` computes local statistics over C_b:

$$\ell_b(c) = \frac{\#\{\, w \in C_b : c \in w \,\}}{|C_b|} \qquad \text{(share of candidates containing letter } c\text{)},$$

$$\pi_b(i, c) = \frac{\#\{\, w \in C_b : w_i = c \,\}}{|C_b|} \qquad \text{(share with letter } c \text{ at position } i\text{)}.$$

Writing $U(w)$ for the set of distinct letters of w, each candidate $w \in C_b$ gets four features in $[0, 1]$:

$$x_1(w) = \frac{\log(1 + n(w))}{\log(1 + \max_{v \in \mathcal{V}} n(v))} \qquad \text{(frequency in the guess file)},$$

$$x_2(w) = \frac{1}{|U(w)|} \sum_{c \in U(w)} \ell_b(c) \qquad \text{(mean letter coverage)},$$

$$x_3(w) = \frac{1}{L} \sum_{i=1}^{L} \pi_b(i, w_i) \qquad \text{(mean positional frequency)},$$

$$x_4(w) = \frac{z(w)}{\max_{v \in \mathcal{V}} z(v)} \qquad \text{(Zipf frequency from } \texttt{wordfreq}\text{)}.$$

Here $n(w)$ is the number of occurrences of w in the guess file as loaded (duplicates are counted; $n(w) = 0$ for $w \notin \mathcal{G}$), and $z(w) = \max(0, \mathrm{zipf}(w))$. If all $n(v)$ or all $z(v)$ are zero, the corresponding feature is 0 everywhere.

Let $\omega_1, \ldots, \omega_4 \ge 0$ be the raw weights (`WEIGHTS`, default $0.2, 0.3, 0.3, 0.2$). If `USE_WORDFREQ` is off or `wordfreq` is not installed, ω_4 is set to 0. The weights are then normalised, $\alpha_j = \omega_j / \sum_i \omega_i$, and the prior is

$$p_b(w) = \frac{s_b(w)}{\sum_{v \in C_b} s_b(v)}, \qquad s_b(w) = \max\Big(0,\ \sum_{j=1}^{4} \alpha_j \, x_j(w)\Big),$$

and $p_b(w) = 0$ for $w \notin C_b$. If all scores are zero, the prior is uniform.

Remark. Since all $x_j \in [0, 1]$ and $\sum_j \alpha_j = 1$, every score satisfies $s_b(w) \le 1$. If every word of $\mathcal{G}$ occurs equally often in the guess file, then $x_1(w) = 1$ for $w \in \mathcal{G}$ and $x_1(w) = 0$ otherwise.

On a board with $C_b \subseteq \mathcal{G}$ the feature x_1 is then constant: it does not change the order of the candidates by prior, but it adds α_1 to every score and thereby flattens the prior,

$$\frac{\max_{w \in C_b} p_b(w)}{\min_{w \in C_b} p_b(w)} \;\le\; \frac{1}{\alpha_1} \qquad (= 5 \text{ with the default weights, 4 without } \texttt{wordfreq}).$$

This changes q_φ, E, P_Σ and therefore the choice of the guess. The prior is a heuristic, not a calibrated probability: x_4 favours common English words, while x_2 and x_3 favour words with typical letters and positions within C_b.

4 Scoring a guess

4.1 Per-board feedback distribution

Fix an active board b with $|C_b| \ge 2$ and a guess $g \in \mathcal{G}'$. The guess partitions C_b into classes

$$C_b^\varphi(g) = \{\, t \in C_b : f(g,t) = \varphi \,\}, \qquad n_\varphi = |C_b^\varphi(g)|, \qquad q_\varphi = \sum_{t \in C_b^\varphi(g)} p_b(t).$$

Here q_φ is the prior probability of observing φ. From this partition we compute three quantities (`pattern_metrics`):

$$H_b(g) = -\sum_{\varphi} q_\varphi \log_2 q_\varphi \qquad \text{(InfoGain: entropy of the feedback)},$$

$$R_b(g) = \sum_{\varphi \neq \mathbf{2}} q_\varphi \, n_\varphi \qquad \text{(ERest: expected number of remaining candidates)},$$

$$E_b(g) = \sum_{\varphi \neq \mathbf{2},\ n_\varphi > 0} q_\varphi \log_2 n_\varphi \qquad \text{(ELog: expected } \log_2 \text{ of the remaining set size)}.$$

The green class is excluded from R_b and E_b because the board is solved in that case. Singletons ($|C_b| = 1$) contribute $H_b = R_b = E_b = 0$: a wrong guess cannot teach anything about a board whose target is already known.

The global scores are sums over the active boards:

$$H(g) = \sum_{b \in \mathcal{B}} H_b(g), \qquad R(g) = \sum_{b \in \mathcal{B}} R_b(g), \qquad E(g) = \sum_{b \in \mathcal{B}} E_b(g).$$

4.2 Relation between ELog and information gain

Proposition 1. *If the prior on board b is uniform, $p_b(t) = 1/|C_b|$, then*

$$E_b(g) = \log_2 |C_b| - H_b(g).$$

Hence, for uniform priors on all boards, minimising $E(g)$ is equivalent to maximising $H(g)$.

Proof. With a uniform prior, $q_\varphi = n_\varphi / |C_b|$, so

$$H_b(g) = -\sum_{\varphi} q_\varphi \log_2 \frac{n_\varphi}{|C_b|} = \log_2 |C_b| - \sum_{\varphi} q_\varphi \log_2 n_\varphi.$$

The green class has $n_{\mathbf{2}} \le 1$ and therefore contributes $\log_2 1 = 0$ to the last sum, so leaving it out does not change the value. The last sum is exactly $E_b(g)$. The term $\log_2 |C_b|$ does not depend on g. □

With a non-uniform prior the two criteria differ: H_b measures how unpredictable the feedback is, whereas E_b is the prior-weighted expectation of $\log_2$ of the number of candidates left after the guess. If the remaining candidates were equally likely, $\log_2 n_\varphi$ would be the number of bits still needed to identify the target; E sums this quantity over all active boards. Under the non-uniform prior this interpretation is an approximation.

4.3 Candidate features

In addition to the information measures, every guess gets direct-solve features:

$$
\begin{aligned}
N(g) &= \#\{\, b \in \mathcal{B} : g \in C_b \,\} && \text{(boards where } g \text{ is a candidate)},\\
S(g) &= \#\{\, b \in \mathcal{B} : C_b = \{g\} \,\} && \text{(singleton boards solved by } g),\\
P_\Sigma(g) &= \sum_{b \in \mathcal{B}} p_b(g) && \text{(expected number of boards solved directly)},\\
P_\exists(g) &= 1 - \prod_{b \in \mathcal{B}} (1 - p_b(g)) && \text{(probability to solve at least one board)},\\
\Lambda(g) &= \sum_{b \in \mathcal{B}} \sum_{c \in U(g)} \ell_b(c) && \text{(letter coverage)},\\
\Pi(g) &= \sum_{b \in \mathcal{B}} \sum_{i=1}^{L} \pi_b(i, g_i) && \text{(position coverage)}.
\end{aligned}
$$

P_Σ follows from linearity of expectation. $P_\exists$ treats the boards as independent, which is an approximation (see Section 8). A guess with $N(g) > 0$ is a *candidate guess*, otherwise a *probe guess*.

Remark. If every candidate set contains its target ($t_b \in C_b$ for all $b \in \mathcal{B}$), then $S(g) \leq 1$: two boards with $C_b = C_{b'} = \{g\}$ would force $t_b = t_{b'} = g$, contradicting distinct targets. Without this assumption, for instance when a target lies outside $\mathcal{A}$ on a board that has not been rebuilt yet, $S(g) \geq 2$ is possible.

5 Decision rule

Let lex min denote the minimum under lexicographic order of tuples. With the default settings (`PREFER_SINGLETON_BOARDS`, `SINGLETON_HARD_PRIORITY` and `USE_ATTEMPT_ORIENTED_SELECTION` all enabled), `choose_next_guess` implements:

Algorithm 1 Choice of the next guess

Require: active boards $\mathcal{B}$, candidate sets C_b, scores for all $g \in \mathcal{G}'$

1: **if** some $b \in \mathcal{B}$ has $|C_b| = 1$ **and** $\mathcal{G}'_S = \{g \in \mathcal{G}' : S(g) > 0\} \neq \emptyset$ **then**
2: **return** $\operatorname{lex\,min}_{g \in \mathcal{G}'_S} \big(-S(g),\ E(g),\ -P_\Sigma(g),\ -N(g),\ -H(g),\ -\Lambda(g),\ -\Pi(g)\big)$
3: **end if**
4: $g^\star \leftarrow \operatorname{lex\,min}_{g \in \mathcal{G}'} \big(E(g),\ -P_\Sigma(g),\ -N(g),\ -P_\exists(g),\ -H(g),\ -\Lambda(g),\ -\Pi(g),\ -x_4(g)\big)$
5: **if** $g^\star$ is a probe **and** margin $m > 0$ **then**
6: $g^c \leftarrow \operatorname{lex\,min}_{g \in \mathcal{G}',\ N(g) > 0} \big(E(g),\ -P_\Sigma(g),\ -N(g),\ -S(g)\big)$
7: **if** $E(g^c) \leq E(g^\star) + m$ **then**
8: **return** g^c
9: **end if**
10: **end if**
11: **return** $g^\star$

The singleton rule reflects that a known board must be entered at some point anyway. Among the singleton words, the one with the smallest E on the remaining boards is chosen. The margin m (`ATTEMPT_REST_MARGIN`, default 0) can prefer a guess that might solve a board over a slightly more informative probe; with the default $m = 0$, lines 5–10 have no effect. If `USE_ATTEMPT_ORIENTED_SELECTION` is disabled, the code uses a different, candidate-first ranking (`rank_candidate_guesses`); this variant is not described here.

6 Bounds on the number of guesses

Proposition 2. *Suppose all M boards are solved. Let w_b be the number of guesses among $g_1, \dots, g_{\tau_b}$ that solved no board. Then at least M guesses were made, and*

$$\sum_{b=1}^{M} \tau_b = \frac{M(M+1)}{2} + \sum_{b=1}^{M} w_b, \qquad \bar{\tau} = \frac{1}{M} \sum_{b=1}^{M} \tau_b \;\geq\; \frac{M+1}{2} = 16.5.$$

Proof. A guess g solves board b only if $g = t_b$. Since the targets are pairwise distinct, each guess solves at most one board, so at least M guesses are needed and the solve indices τ_b are pairwise distinct. Order the boards so that $\tau_{(1)} < \cdots < \tau_{(M)}$. Among $g_1, \dots, g_{\tau_{(j)}}$ exactly j guesses solve a board, namely those at $\tau_{(1)}, \dots, \tau_{(j)}$; the others solve none. Hence $\tau_{(j)} = j + w_{(j)}$. Summing over j gives the identity, and $w_b \geq 0$ gives the bound. □

Thus every guess that solves no board (every probe guess and every candidate guess that misses) pushes all later solve indices back by one, and $\bar{\tau} - \frac{M+1}{2} = \frac{1}{M} \sum_b w_b$. Equality $\bar{\tau} = 16.5$ holds exactly when every guess up to the last solve solves a board. The score $\bar{\tau}$ therefore measures how quickly the solver moves from exploration to clearing boards.

7 Computational cost

One decision evaluates the feedback of every guess in $\mathcal{G}'$ against every candidate on every active non-singleton board:

$$\text{pairs per turn} = |\mathcal{G}'| \cdot \sum_{b \in \mathcal{B},\ |C_b| \geq 2} |C_b|.$$

At the first decision every board has $C_b = \mathcal{A}$, which gives $M \cdot |\mathcal{G}'| \cdot |\mathcal{A}|$ pairs ($M \cdot |\mathcal{G}| \cdot |\mathcal{A}|$ without rejected words), and $M \cdot |\mathcal{G}'| \cdot |\mathcal{G}|$ with `--wide-solutions`, where $\mathcal{A} = \mathcal{G}$. The pair counter of the GPU backend equals this formula. The CPU backend also counts one pair per active singleton board, i.e. it reports $|\mathcal{G}'| \cdot \sum_{b \in \mathcal{B}} |C_b|$; both agree whenever no active board is a singleton, in particular at the first decision.

The bridge caches every decision under a key built from a fingerprint (solver file, guess file, solution file, weights, `wordfreq` settings), the history and the list of rejected words. As long as these inputs do not change, the first decision is computed only once and served from the cache afterwards.

8 Limitations

- **Greedy.** The rule looks one guess ahead. It does not plan sequences of guesses, so it is not guaranteed to be optimal.
- **Independent boards.** Each board is modelled separately. A guessed word is removed from every board that did not show all green for it (Section 2), so a word that solved one board never remains a candidate elsewhere. The remaining coupling through distinct targets ($t_b \neq t_{b'}$) is ignored: if $C_b = \{w\}$ on an active board b (target known but not yet entered), w is not removed from $C_{b'}$ for $b' \neq b$, and more generally the joint constraint between active boards is not used. This can overstate $N(g)$, $P_\Sigma(g)$ and $P_\exists(g)$ and distort $E(g)$. $P_\exists$ additionally assumes independence.
- **Heuristic prior.** The weights α_j are fixed constants and are not fitted to real target distributions. Fitting them to the logs in `output/live_games.jsonl` is a natural next step.

- **Unknown solution list.** The prior only covers $\mathcal{A}$ until a board is rebuilt. In rebuilt boards, x_4 (Zipf frequency) is the only feature that measures how common a word is in English; x_1 only reflects the guess file, and x_2 and x_3 only describe letter patterns.
- **Guess list.** Only words in $\mathcal{G}'$ are proposed. A target outside $\mathcal{G}'$ (for example a word of $\mathcal{A} \setminus \mathcal{G}$) can never be entered by the solver.